

综合实践报告

课程: 机器学习	实验名称: 基于朴素贝叶斯的文本分类	日期: 4.22
姓名: 雷文豪	学号: 138022024031	专业班级: 人工智能

1 实验目的

朴素贝叶斯算法是有监督学习算法，目的是解决分类问题。该算法的优点在于简单易懂、学习效率高，在某些领域的分类问题中能够与决策树、神经网络相媲美。但由于该算法假设自变量之间相互独立，且处理连续变量时假定特征数值服从正态分布，易导致算法精度在某种程度上受到影响。本次实验的目的在于通过完成不当言论过滤、垃圾邮件分类、以及新闻分类三个子任务，加深对朴素贝叶斯算法的理解。

2 实验内容提要与实验设备

1. 针对待解决问题设计适合的求解方法。
2. 对给定数据集，设计求解过程、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

3.1 实验内容 1（言论过滤器）

为了不妨碍社区的健康发展，我们需要对侮辱性言论加以屏蔽，因此要设计一个快速过滤器：一旦某条留言中出现负面或侮辱性的词语，就将其标注为内容不当。这类内容过滤是非常普遍的需求。针对这个问题，定义两个类别：用 1 表示侮辱类，用 0 表示非侮辱类。

我们可以将文本视为单词向量或词条向量，即，把句子转换成向量的形式。首先，考察所有文档中出现的单词，然后决定哪些单词被纳入词汇表（即所需的单词集合），接着需要将每一篇文档都转换为基于该词汇表的向量。为简化起见，这里先假设文本已经被切分好并存入列表，同时也已完成对词向量的分类标注，代码如下：

```

1 def loadDataSet():
2     postingList=[['my', 'dog', 'has', 'flea', 'problems', 'help', 'please'],
3                   ['maybe', 'not', 'take', 'him', 'to', 'dog', 'park', 'stupid'],
4                   ['my', 'dalmation', 'is', 'so', 'cute', 'I', 'love', 'him'],
5                   ['stop', 'posting', 'stupid', 'worthless', 'garbage'],
6                   ['mr', 'licks', 'ate', 'my', 'steak', 'how', 'to', 'stop', 'him'],
7                   ['quit', 'buying', 'worthless', 'dog', 'food', 'stupid']]
8     classVec = [0,1,0,1,0,1]
9     return postingList, classVec

```

有了待训练的文本及对应的类别之后，接下来，我们需要创建一个词汇表，并将切分好的词条转换为词条向量。代码框架如下，请填写完整。

```

1 def createVocabList(dataSet):
2     # 函数说明：将切分的实验样本词条整理成不重复的词条列表（即词汇表）
3     vocabSet = set([])
4
5     [Put your code here...]
6     # 遍历每篇文档
7     for document in dataSet:
8         # 取并集，去重
9         vocabSet = vocabSet | set(document)
10        # 返回为列表
11        vocabSet = list(vocabSet)
12
13    return vocabSet
14
15 def setOfWords2Vec(vocabList, inputSet):
16     # 函数说明：根据 vocabList 词汇表，将 inputSet 向量化，向量的每个元素为 1 或 0
17     returnVec = [0] * len(vocabList)
18
19    [Put your code here...]
20    # 遍历输入的词
21    for word in inputSet:
22        if word in vocabList:
23            returnVec[vocabList.index(word)] = 1
24
25    return returnVec

```

得到词条向量后，接下来，就可以通过词条向量来训练朴素贝叶斯分类器，代码框架如下，请填写完整。

```
1  def trainNB(trainMatrix, trainCategory):
2      """
3      参数说明：
4      trainMatrix - 训练文档矩阵，即 setOfWords2Vec 返回的 returnVec 构成的矩阵
5      trainCategory - 训练类别标签向量，即 loadDataSet 返回的 classVec
6      """
7      numTrainDocs = len(trainMatrix)
8      numWords = len(trainMatrix[0])
9
10     pAbusive = sum(trainCategory) / numTrainDocs # 文档属于侮辱类的概率
```

```

12     p0Num = np.zeros(numWords); p1Num = np.zeros(numWords)
13     p0Denom = 0.0; p1Denom = 0.0
14
15     p1Vect = [Put your code here...] # 侮辱类的每个词的条件概率数组
16     p0Vect = [Put your code here...] # 非侮辱类的每个词的条件概率数组
        # 直接做拉普拉斯平滑处理
        p0Num = np.ones(numWords)
        p1Num = np.ones(numWords)

        p0Denom = 32
        p1Denom = 32

        for i in range(numTrainDocs):
            if trainCategory[i] == 1:
                # 计算侮辱类的词频
                p1Num += trainMatrix[i]
                p1Denom += sum(trainMatrix[i])
                # 计算非侮辱类的词频
            else:
                p0Num += trainMatrix[i]
                p0Denom += sum(trainMatrix[i])

        p1Vect = np.log(p1Num / p1Denom)
        p0Vect = np.log(p0Num / p0Denom)
17     return p0Vect, p1Vect, pAbusive

```

通过上述代码，我们可以训练好朴素贝叶斯分类器。接下来，使用该分类器进行分类，代码框架如下，请填写完整。

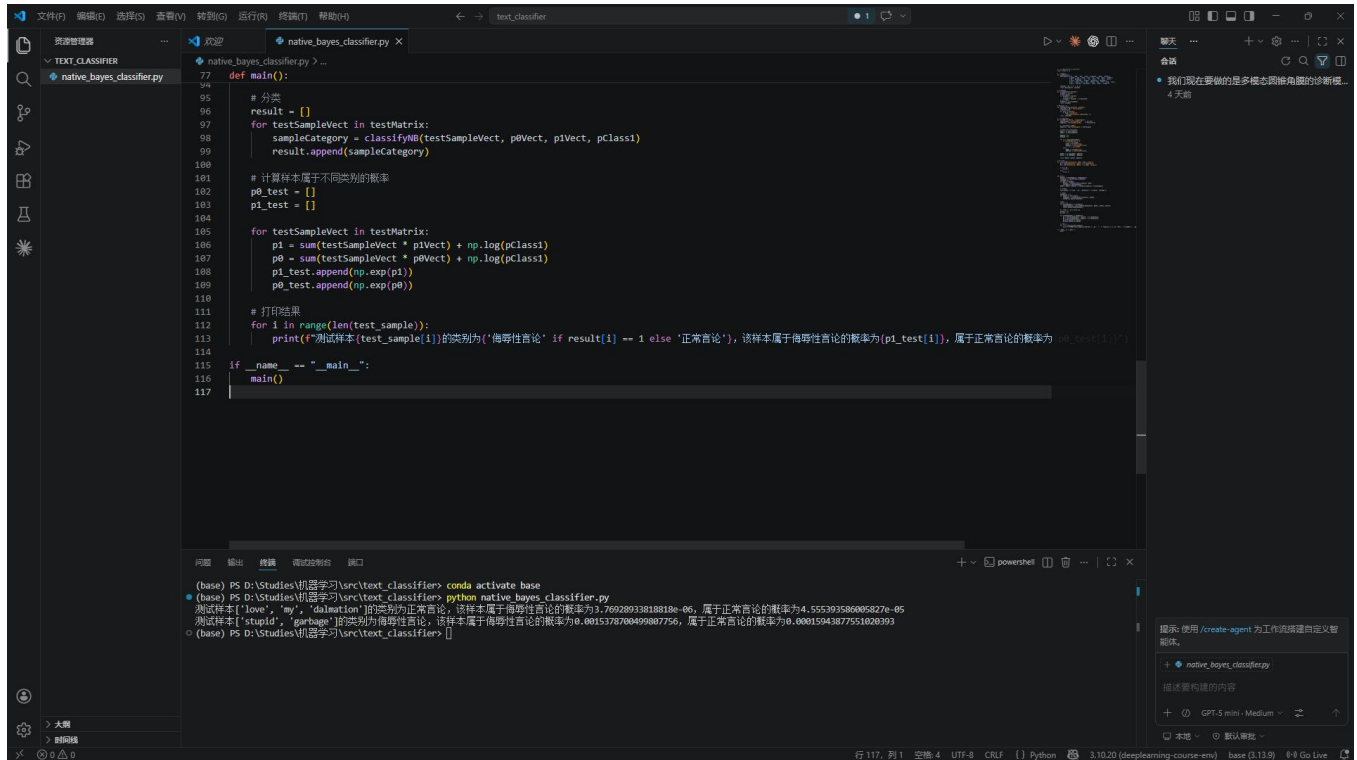
```

1     def classifyNB(vec2Classify, p0Vec, p1Vec, pClass1):
2         """
3         参数说明：
4         vec2Classify - 待分类的词条数组
5         p0Vec        - 侮辱类的条件概率数组
6         p1Vec        - 非侮辱类的条件概率数组
7         pClass1      - 文档属于侮辱类的概率
8         """
9
10        [Put your code here...]
        p1 = sum(vec2Classify * p1Vec) + np.log(pClass1)
        p0 = sum(vec2Classify * p0Vec) + np.log(1 - pClass1)
11
12        if p1 > p0:
13            return 1

```

```
14     else:
15         return 0
```

通过完成上述代码，对测试样本 ['love', 'my', 'dalmation'] 和 ['stupid', 'garbage'] 进行分类，并呈现实验结果。



```
def main():
    # 分类
    result = []
    for testSampleVect in testMatrix:
        sampleCategory = classifyNB(testSampleVect, p0Vect, p1Vect, pClass1)
        result.append(sampleCategory)

    # 计算样本属于不同类别的概率
    p0_test = []
    p1_test = []

    for testSampleVect in testMatrix:
        p1 = sum(testSampleVect * p1Vect) + np.log(pClass1)
        p0 = sum(testSampleVect * p0Vect) + np.log(pClass1)
        p1_test.append(np.exp(p1))
        p0_test.append(np.exp(p0))

    # 打印结果
    for i in range(len(test_sample)):
        print(f"测试样本[test_sample[{i}]]的类别为('侮辱性言论' if result[i] == 1 else '正常言论'), 该样本属于侮辱性言论的概率为(p1_test[{i}]), 属于正常言论的概率为(p0_test[{i}])")

if __name__ == "__main__":
    main()
```

输出结果:

```
(base) PS D:\Studies\机器学习\src\text_classifier> python native_bayes_classifier.py
测试样本['love', 'my', 'dalmation']的类别为正常言论, 该样本属于侮辱性言论的概率为3.76928033818818e-06, 属于正常言论的概率为4.555393586005827e-05
测试样本['stupid', 'garbage']的类别为侮辱性言论, 该样本属于侮辱性言论的概率为0.0015378700499807756, 属于正常言论的概率为0.00015943877551028393
(base) PS D:\Studies\机器学习\src\text_classifier>
```

课堂作业. 通过观察实验结果，你发现了什么问题？如何改进？请给出你的改进方法，并同时给出改进后的代码和实验结果。

3.2 实验内容 2（垃圾邮件过滤）

在第3.1节的示例中，我们初步认识了如何用朴素贝叶斯分类器来处理简单的文本任务。本节将介绍朴素贝叶斯最为著名的应用之一：电子垃圾邮件过滤。使用朴素贝叶斯对电子邮件进行分类的具体步骤如下，请根据以下步骤完成相应的任务。

- **收集数据：**提供文本文件，见 email 下的 spam 和 ham 两个文件夹，其中 spam 内的文档为垃圾邮件。
- **准备数据：**

1. 将文本文件转化为词条列表。对于英文文本，以非字母、非数字作为符号，使用 Python 的 `re.split()`，配合正则表达式进行切分。同时，把除了单个字母（例如大写的 I）以外的其它单词变成小写。
 2. 存储每个文本文件的标签。
 3. 构建词汇表。
- **分析数据：**检查词条以确保解析正确。
 - **训练算法：**使用之前定义的 `trainNB()` 函数，将文档划分为训练集和测试集，并用训练集来训练朴素贝叶斯模型。
 - **测试算法：**调用 `classifyNB()`，编写测试函数来计算模型在测试集上的错误率。
 - **使用算法：**构建一个完整程序，对一组文档进行分类，输出分类错误的文档。

1. 完整代码实现

```
import os

import re

import numpy as np

from sklearn.model_selection import train_test_split, StratifiedKFold

# 文本读取

def textParse(text):

    # 分隔文本

    words = re.split(r'[^a-zA-Z0-9]', text)

    # 做过滤

    new_words = []

    for word in words:

        if len(word) == 0:

            continue

        if len(word) == 1:

            new_words.append(word)

        else:

            new_words.append(word.lower())
```

```
    return new_words
```

```
# 读取数据
```

```
def loadEmailData(folder_path):  
    word_list = []  
    for filename in os.listdir(folder_path):  
        if filename.endswith(".txt"):  
            path = os.path.join(folder_path, filename)  
            with open(path, "r", encoding="latin1") as f:  
                content = f.read()  
                words = textParse(content)  
                word_list.append(words)  
    return word_list
```

```
# 创建词汇表
```

```
def createVocabList(dataSet):  
    vocabSet = set([])  
    # 遍历每篇文档  
    for document in dataSet:  
        # 取并集，去重  
        vocabSet = vocabSet | set(document)  
    # 返回为列表  
    vocabSet = list(vocabSet)  
    return vocabSet
```

```
# 词条向量转换
```

```
def setOfWords2Vec(vocabList, inputSet):  
    returnVec = [0] * len(vocabList)  
    # 遍历输入的词
```

```
for word in inputSet:

    if word in vocabList:

        returnVec[vocabList.index(word)] = 1

return returnVec
```

训练朴素贝叶斯

```
def trainNB(trainMatrix, trainCategory):

    numTrainDocs = len(trainMatrix)    # 文本数量

    numWords = len(trainMatrix[0])     # 词条向量长度

    # 侮辱类的先验概率统计

    pAbusive = sum(trainCategory) / numTrainDocs

    # 直接做拉普拉斯平滑处理

    p0Num = np.ones(numWords)

    p1Num = np.ones(numWords)

    p0Denom = len(trainMatrix)

    p1Denom = len(trainMatrix)

    for i in range(numTrainDocs):

        if trainCategory[i] == 1:

            # 计算侮辱类的词频

            p1Num += trainMatrix[i]

            p1Denom += sum(trainMatrix[i])

            # 计算非侮辱类的词频

        else:

            p0Num += trainMatrix[i]

            p0Denom += sum(trainMatrix[i])
```

```
p1Vect = np.log(p1Num / p1Denom)

p0Vect = np.log(p0Num / p0Denom)


return p0Vect, p1Vect, pAbusive


# 分类函数
def classifyNB(vec2Classify, p0Vec, p1Vec, pClass1):

    p1 = sum(vec2Classify * p1Vec) + np.log(pClass1)

    p0 = sum(vec2Classify * p0Vec) + np.log(1 - pClass1)


    if p1 > p0:

        return 1

    else:

        return 0


def main():

    spam_folder = r"email\spam"

    ham_folder = r"email\ham"


    spam_words = loadEmailData(spam_folder)

    ham_words = loadEmailData(ham_folder)


    all_data = spam_words + ham_words

    all_labels = [1] * len(spam_words) + [0] * len(ham_words)


    X_train, X_test, y_train, y_test = train_test_split(

        all_data,

        all_labels,
```

```
        test_size=0.2,

        random_state=42,

        stratify=all_labels

    )

vocabList = createVocabList(all_data)

train_mat = np.array([setOfWords2Vec(vocabList, d) for d in X_train])

test_mat = np.array([setOfWords2Vec(vocabList, d) for d in X_test])

y_train_np = np.array(y_train)

y_test_np = np.array(y_test)


kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

fold_accs = []

for fold, (train_idx, val_idx) in enumerate(kf.split(train_mat, y_train_np)):

    X_k_train, X_k_val = train_mat[train_idx], train_mat[val_idx]

    y_k_train, y_k_val = y_train_np[train_idx], y_train_np[val_idx]


    p0v, plv, pA = trainNB(X_k_train, y_k_train)

    correct = 0

    for i in range(len(X_k_val)):

        if classifyNB(X_k_val[i], p0v, plv, pA) == y_k_val[i]:

            correct += 1

    acc = correct / len(X_k_val)

    fold_accs.append(acc)

    print(f"第{fold + 1}折准确率: {acc:.2%}")


print(f"\n5折平均准确率: {np.mean(fold_accs):.2%}")


p0_vec, pl_vec, p_ab = trainNB(train_mat, y_train_np)
```

```
correct = 0

for i in range(len(test_mat)):

    if classifyNB(test_mat[i], p0_vec, p1_vec, p_ab) == y_test_np[i]:

        correct += 1

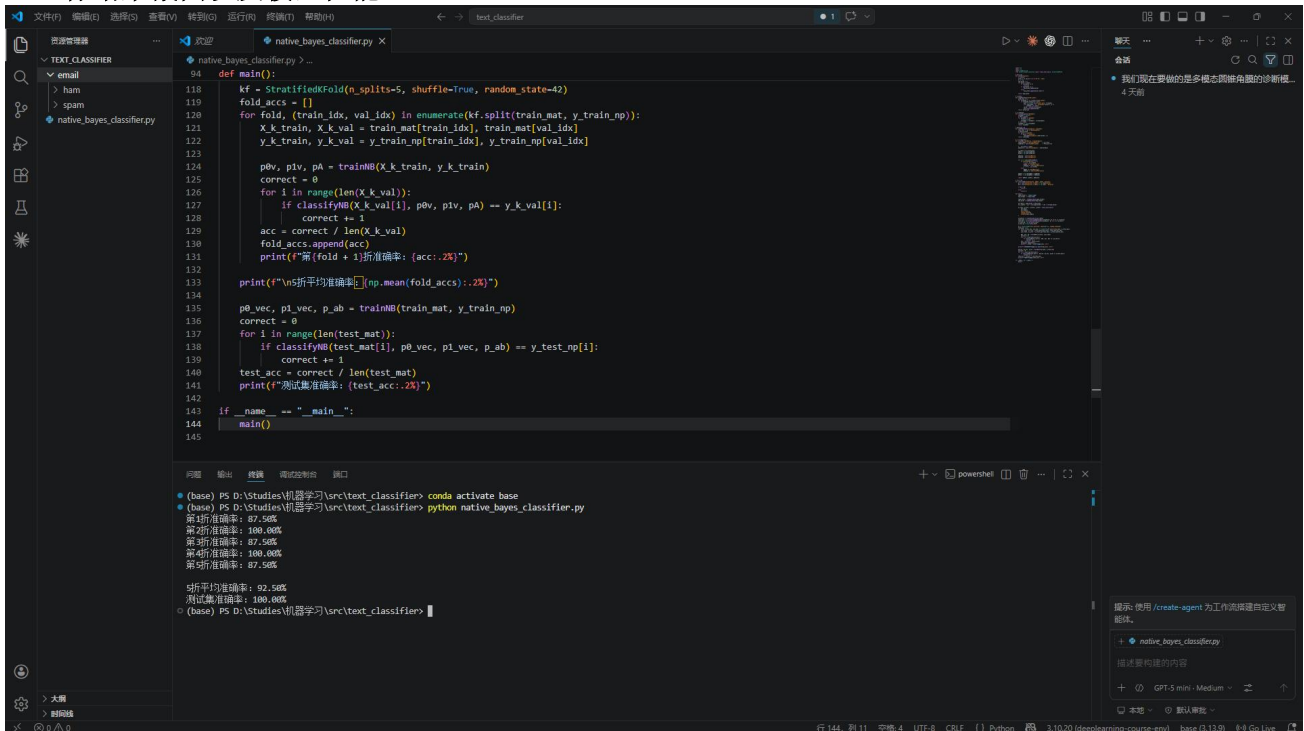
test_acc = correct / len(test_mat)

print(f"测试集准确率: {test_acc:.2%}")

if __name__ == "__main__":

    main()
```

2. 运行结果截图以及模型性能



3.3 实验内容 3（新闻分类）

在第3.2节中，我们完成了对英文语句的处理。本节继续探讨对中文新闻文档的处理，并使用朴素贝叶斯分类模型对新闻的类别进行分类；其中，最重要的是如何对中文句子进行切分。幸运的是，Python的第三方库jieba已经可以帮我们完成这项工作。接下来，请按照如下步骤完成新闻分类的任务。

1. pip install jieba 安装 jieba; 教程请看 <https://github.com/fxsjy/jieba>。
2. 准备数据集 SogouC。
3. 编写代码，对每个文档的中文语句进行切分，转换成词条列表，并划分训练集和测试集。
4. 对训练集中的所有单词进行词频统计，并按降序排序。

5. 进行特征选择。规则如下：去掉高频词（暂时定为 100）；去除数字；使用 stopwords_cn.txt 去除一些特定的词语，如：“的”、“在”、“不”、“当然”等这类对新闻分类无影响的介词、代词、连词等。
6. 基于训练集，使用 Sklearn 的 MultinomialNB 函数构建朴素贝叶斯分类器，并使用测试集进行测试。

随堂思考： 如果需要确定去除多少高频词是合适的，我们应该如何做？请写出你的解决方案。

1. 完整代码实现

```
import os
import re
import jieba
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
from collections import Counter
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
import warnings
warnings.filterwarnings('ignore')

# 加载停用词表
def load_stopwords(stopwords_path='stopwords_cn.txt'):
    stopwords = set()

    # 检查文件是否存在
    if os.path.exists(stopwords_path):
        try:
            with open(stopwords_path, 'r', encoding='utf-8') as f:
                for line in f:
                    word = line.strip()
                    if word:
                        stopwords.add(word)
            print(f"成功加载停用词文件：{stopwords_path}，共 {len(stopwords)} 个停用词")
        except Exception as e:
            print(f"读取停用词文件失败：{e}")
            stopwords = create_default_stopwords()
    else:
        print(f"停用词文件不存在：{stopwords_path}，使用默认停用词")
        stopwords = create_default_stopwords()

    return stopwords

def create_default_stopwords():
```



```

        class_mapping[class_code] = class_name
    print(f"成功加载类别映射文件: {class_list_file}")
    print(f"识别到 {len(class_mapping)} 个类别: {list(class_mapping.values())}")
except Exception as e:
    print(f"读取类别映射文件失败: {e}")
    print("将尝试直接使用文件夹名称作为类别")
    class_mapping = None
else:
    print(f"类别映射文件不存在: {class_list_file}")
    print("将尝试直接使用文件夹名称作为类别")
    class_mapping = None

documents = []
labels = []
label_counts = Counter()

# 遍历根目录下的所有子文件夹
if not os.path.exists(data_root_dir):
    print(f"数据根目录不存在: {data_root_dir}")
    return None, None

# 获取所有类别文件夹
if class_mapping:
    # 使用类别映射
    class_folders = [os.path.join(data_root_dir, class_code) for class_code in
class_mapping.keys()]
    class_names = list(class_mapping.values())
else:
    # 直接使用文件夹名称
    class_folders = [os.path.join(data_root_dir, folder) for folder in
os.listdir(data_root_dir)
                    if os.path.isdir(os.path.join(data_root_dir, folder))]
    class_names = [os.path.basename(folder) for folder in class_folders]

print(f"正在扫描以下类别文件夹:")
for folder, class_name in zip(class_folders, class_names):
    print(f"    - {class_name}: {folder}")

# 读取每个类别文件夹中的文档
for class_idx, (class_folder, class_name) in enumerate(zip(class_folders,
class_names)):
    if not os.path.exists(class_folder):
        print(f"类别文件夹不存在: {class_folder}, 跳过")
        continue

    print(f"\n处理类别: {class_name} ({class_idx+1}/{len(class_folders)})")

# 获取该类别下的所有.txt文件
txt_files = [f for f in os.listdir(class_folder) if f.endswith('.txt')]

```

```

print(f"找到 {len(txt_files)} 个文本文件")

# 使用tqdm显示进度条
for filename in tqdm(txt_files, desc=f'处理 {class_name}', leave=False):
    file_path = os.path.join(class_folder, filename)
    try:
        # 尝试不同的编码方式
        encodings = ['utf-8', 'gbk', 'gb2312', 'latin1']
        content = None

        for encoding in encodings:
            try:
                with open(file_path, 'r', encoding=encoding) as f:
                    content = f.read()
                break # 成功读取, 跳出循环
            except UnicodeDecodeError:
                continue

        if content is None:
            print(f"无法读取文件 {file_path}, 跳过")
            continue

        # 文本清理
        content = clean_text(content)

        # 只保留有一定长度的文档
        if len(content) > 20: # 至少20个字符
            documents.append(content)
            labels.append(class_name)
            label_counts[class_name] += 1

    except Exception as e:
        print(f"处理文件 {file_path} 时出错: {e}")
        continue

print(f"\n数据加载完成!")
print(f"总共加载 {len(documents)} 个文档, {len(set(labels))} 个类别")

# 显示类别分布
print("\n类别分布统计:")
for class_name, count in label_counts.most_common():
    percentage = count / len(documents) * 100
    print(f"    {class_name}: {count} 篇 ({percentage:.1f}%)")

return documents, labels

def clean_text(text):
    # 移除HTML标签

```

```

text = re.sub(r'<[^>]+>', '', text)
# 移除特殊字符，保留中英文、数字和基本标点
text = re.sub(r'[\u4e00-\u9fa5a-zA-Z0-9,。！？；：\'"“”‘’（）【】《》\-\s]',
'', text)
# 替换多余空白
text = re.sub(r'\s+', ' ', text)
# 移除首尾空白
text = text.strip()
return text

```

中文分词和预处理

```

def chinese_segmentation(text, stopwords):
    # 使用jieba进行精确分词
    words = jieba.lcut(text)

    # 过滤条件
    filtered_words = []
    for word in words:
        word = word.strip()
        # 跳过空字符串
        if not word:
            continue

        # 跳过停用词
        if word in stopwords:
            continue

        # 跳过单个字符（除了中文单字）
        if len(word) == 1 and not '\u4e00' <= word <= '\u9fa5':
            continue

        # 跳过纯数字
        if re.match(r'^\d+$', word):
            continue

        # 跳过URL、邮箱等
        if re.match(r'http[s]?://|www\.', word) or '@' in word:
            continue

        # 保留有意义的词
        if len(word) >= 1: # 允许中文单字
            filtered_words.append(word)

    return filtered_words

# 构建特征向量
def build_feature_matrix(documents, stopwords, remove_top_n=100, max_features=5000):
    print("\n开始构建特征矩阵")

```

```
# 分词处理
print("进行中文分词")
tokenized_docs = []
all_words = []

for doc in tqdm(documents, desc='分词处理', leave=False):
    words = chinese_segmentation(doc, stopwords)
    tokenized_docs.append(' '.join(words))
    all_words.extend(words)

# 词频统计
print("进行词频统计")
word_counter = Counter(all_words)
print(f"总词汇量: {len(word_counter)}")

# 显示高频词
print("词频统计前20名:")
for i, (word, count) in enumerate(word_counter.most_common(20), 1):
    print(f"      {i:2d}. {word}: {count}次")

# 特征选择: 去除高频词
print(f"去除前 {remove_top_n} 个高频词")
sorted_words = sorted(word_counter.items(), key=lambda x: x[1], reverse=True)
high_freq_words = set([word for word, _ in sorted_words[:remove_top_n]])

# 构建词汇表
vocab = [word for word in word_counter.keys() if word not in high_freq_words]
print(f"特征选择后词汇量: {len(vocab)}")

# 限制最大特征数
if len(vocab) > max_features:
    print(f"限制特征数量到 {max_features}")
    # 按词频排序, 取前max_features个
    vocab = [word for word, _ in sorted(word_counter.items(), key=lambda x: x[1],
reverse=True)
            if word not in high_freq_words][:max_features]

# 创建特征向量
print("创建特征向量")
vectorizer = CountVectorizer(
    vocabulary=vocab,
    min_df=2, # 至少在2个文档中出现
    max_df=0.85, # 最多在85%的文档中出现
    token_pattern=r'(?u)\b\w+\b' # 匹配所有词
)

X = vectorizer.fit_transform(tokenized_docs)
```

```
print(f"特征矩阵形状: {X.shape}")

return X, vectorizer, word_counter

# 寻找最佳高频词去除数量
def find_optimal_high_freq_removal(documents, labels, stopwords, test_size=0.2,
random_state=42):
    print("""
    解决方案: 网格搜索 + 交叉验证
    1. 定义候选的去除数量范围
    2. 对每个数量进行5折交叉验证
    3. 选择在验证集上平均准确率最高的数量
    4. 使用学习曲线和准确率曲线分析结果
    5. 考虑计算复杂度和模型性能的平衡
    """)

    # 划分训练集和测试集 (只用训练集进行参数搜索)
    X_train, X_test, y_train, y_test = train_test_split(
        documents, labels, test_size=test_size, random_state=random_state,
        stratify=labels
    )
    print(f"参数搜索使用 {len(X_train)} 个训练样本")

    # 候选的去除数量
    removal_counts = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 150, 200]
    cv_scores = []
    train_scores = []

    print(f"\n开始网格搜索, 测试 {len(removal_counts)} 个候选值...")

    for remove_count in tqdm(removal_counts, desc='网格搜索'):
        try:
            # 构建特征矩阵
            X_train_vec, vectorizer, _ = build_feature_matrix(
                X_train, stopwords, remove_top_n=remove_count, max_features=3000
            )

            # 5折交叉验证
            clf = MultinomialNB()
            kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=random_state)
            scores = cross_val_score(clf, X_train_vec, y_train, cv=kf,
            scoring='accuracy')

            avg_score = np.mean(scores)
            cv_scores.append(avg_score)

            # 训练集准确率
            clf.fit(X_train_vec, y_train)
            train_acc = clf.score(X_train_vec, y_train)
```

```

        train_scores.append(train_acc)

    print(f"去除 {remove_count:3d} 个高频词: 5折CV准确率 = {avg_score:.4f}, 训练
准确率 = {train_acc:.4f}")

except Exception as e:
    print(f"去除 {remove_count} 个高频词时出错: {e}")
    cv_scores.append(0)
    train_scores.append(0)

# 找到最佳值
best_idx = np.argmax(cv_scores)
best_removal = removal_counts[best_idx]
best_score = cv_scores[best_idx]

print(f"\n最佳结果:")
print(f"最优高频词去除数量: {best_removal}")
print(f"对应的5折CV平均准确率: {best_score:.4f}")

# 可视化分析
plt.figure(figsize=(15, 6))
plt.rcParams['font.sans-serif'] = ['SimHei']

# 准确率曲线
plt.subplot(1, 2, 1)
plt.plot(removal_counts, cv_scores, 'b-o', linewidth=2, label='5折CV准确率')
plt.plot(removal_counts, train_scores, 'r--s', linewidth=2, label='训练准确率')
plt.axvline(x=best_removal, color='g', linestyle='--', label=f'最佳点:
{best_removal}')
plt.xlabel('去除的高频词数量')
plt.ylabel('准确率')
plt.title('高频词去除数量对模型性能的影响')
plt.legend()
plt.grid(True)

# 过拟合分析
plt.subplot(1, 2, 2)
overfitting = np.array(train_scores) - np.array(cv_scores)
plt.plot(removal_counts, overfitting, 'g-^', linewidth=2)
plt.axhline(y=0, color='k', linestyle='-', alpha=0.3)
plt.xlabel('去除的高频词数量')
plt.ylabel('过拟合程度 (训练准确率 - 验证准确率)')
plt.title('过拟合程度分析')
plt.grid(True)

plt.tight_layout()
plt.savefig('high_freq_removal_analysis.png')
print("分析结果已保存为 'high_freq_removal_analysis.png'")

```

```
    return best_removal, best_score

# 7. 主函数
def main():
    # 配置参数
    data_root_dir = 'SogouC'    # 数据根目录
    class_list_file = 'ClassList.txt'    # 类别映射文件
    stopwords_file = 'stopwords_cn.txt'    # 停用词文件
    random_state = 42

    # 加载停用词
    stopwords = load_stopwords(stopwords_file)

    # 加载数据集
    documents, labels = load_news_data(data_root_dir, class_list_file)

    if documents is None or len(documents) == 0:
        print("未加载到有效文档，程序终止")
        return

    best_removal, best_score = find_optimal_high_freq_removal(documents, labels,
stopwords)

    # 划分训练集和测试集
    X_train, X_test, y_train, y_test = train_test_split(
        documents, labels, test_size=0.2, random_state=random_state, stratify=labels
    )
    print(f"训练集: {len(X_train)} 篇, 测试集: {len(X_test)} 篇")

    # 构建特征矩阵
    X_train_vec, vectorizer, word_counter = build_feature_matrix(
        X_train, stopwords, remove_top_n=best_removal, max_features=5000
    )

    # 转换测试集
    X_test_tokenized = [' '.join(chinese_segmentation(doc, stopwords)) for doc in
X_test]
    X_test_vec = vectorizer.transform(X_test_tokenized)

    # 训练朴素贝叶斯分类器
    clf = MultinomialNB()
    clf.fit(X_train_vec, y_train)

    # 评估模型
    # 训练集评估
    train_pred = clf.predict(X_train_vec)
    train_acc = accuracy_score(y_train, train_pred)
    print(f"训练集准确率: {train_acc:.4f}")
```

```

# 测试集评估
test_pred = clf.predict(X_test_vec)
test_acc = accuracy_score(y_test, test_pred)
print(f"测试集准确率: {test_acc:.4f}")

# 详细分类报告
print("详细分类报告")
print(classification_report(y_test, test_pred))

# 分析错误分类
print("错误分类分析")
errors = [(i, actual, pred) for i, (actual, pred) in enumerate(zip(y_test,
test_pred))
            if actual != pred]

print(f"总共 {len(errors)} 个错误分类 ({len(errors)/len(y_test)*100:.1f}%)")

if errors:
    print("\n前10个错误分类示例:")
    for i, (idx, actual, pred) in enumerate(errors[:10], 1):
        doc_preview = ' '.join(X_test[idx].split()[:15]) # 取前15个词
        print(f" {i:2d}. 实际: {actual:<4s} | 预测: {pred:<4s} | 内容:
{doc_preview}...")

# 保存结果
print("保存结果")

# 保存特征重要性
print("保存特征重要性分析")
feature_names = vectorizer.get_feature_names_out()
class_names = sorted(set(labels))

# 为每个类别保存最重要的特征
feature_importance = {}
for i, class_name in enumerate(class_names):
    log_prob = clf.feature_log_prob_[i]
    top_indices = np.argsort(log_prob)[::-1][:10] # 取最重要的10个特征

    important_features = []
    for idx in top_indices:
        feature = feature_names[idx]
        importance = log_prob[idx]
        important_features.append((feature, importance))

    feature_importance[class_name] = important_features

# 保存到文件
with open('feature_importance.txt', 'w', encoding='utf-8') as f:
    f.write("各类别最重要的特征:\n")

```

```

        f.write("="*50 + "\n")
        for class_name, features in feature_importance.items():
            f.write(f"\n{class_name} 类别:\n")
            for i, (feature, importance) in enumerate(features, 1):
                f.write(f"    {i:2d}. {feature}: {importance:.4f}\n")

print("特征重要性已保存到 'feature_importance.txt'")

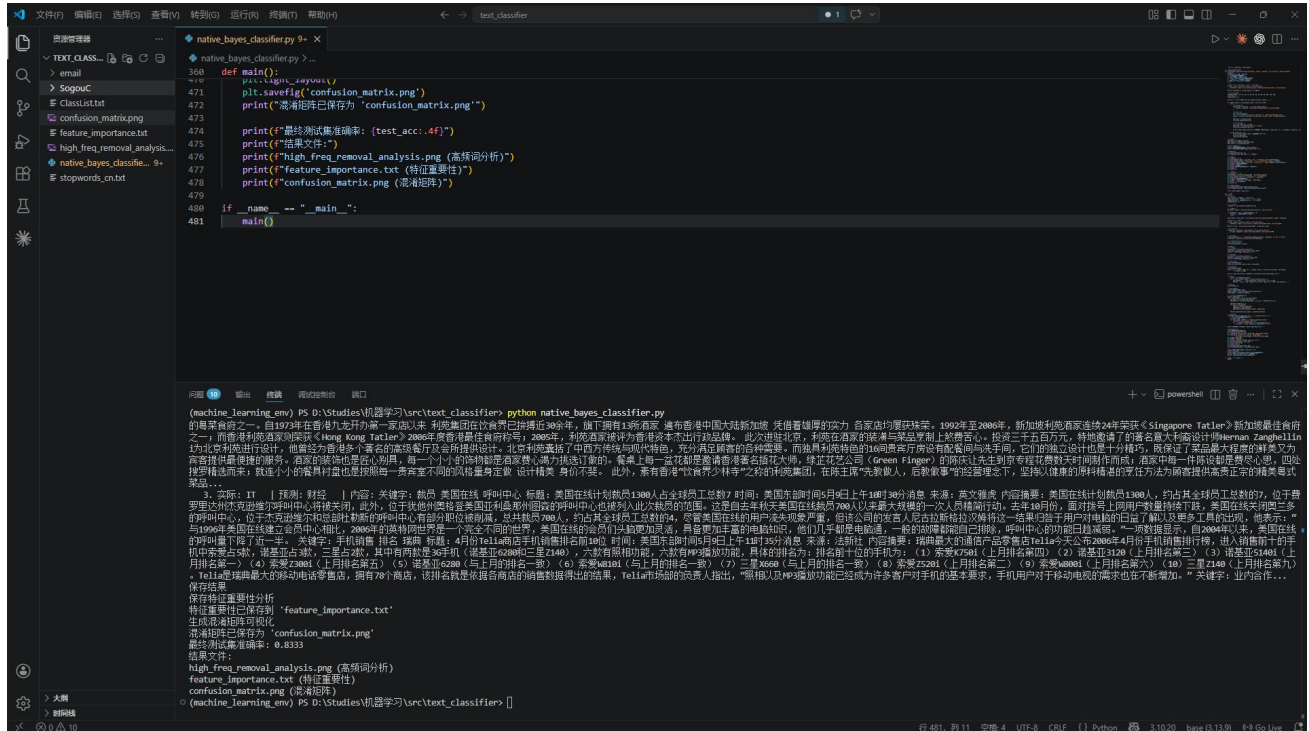
# 可视化混淆矩阵
print("生成混淆矩阵可视化")
plt.figure(figsize=(12, 10))
cm = confusion_matrix(y_test, test_pred, labels=class_names)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel(' 预测类别')
plt.ylabel(' 实际类别')
plt.title(f' 混淆矩阵 (准确率: {test_acc:.4f})')
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.savefig('confusion_matrix.png')
print("混淆矩阵已保存为 'confusion_matrix.png'")

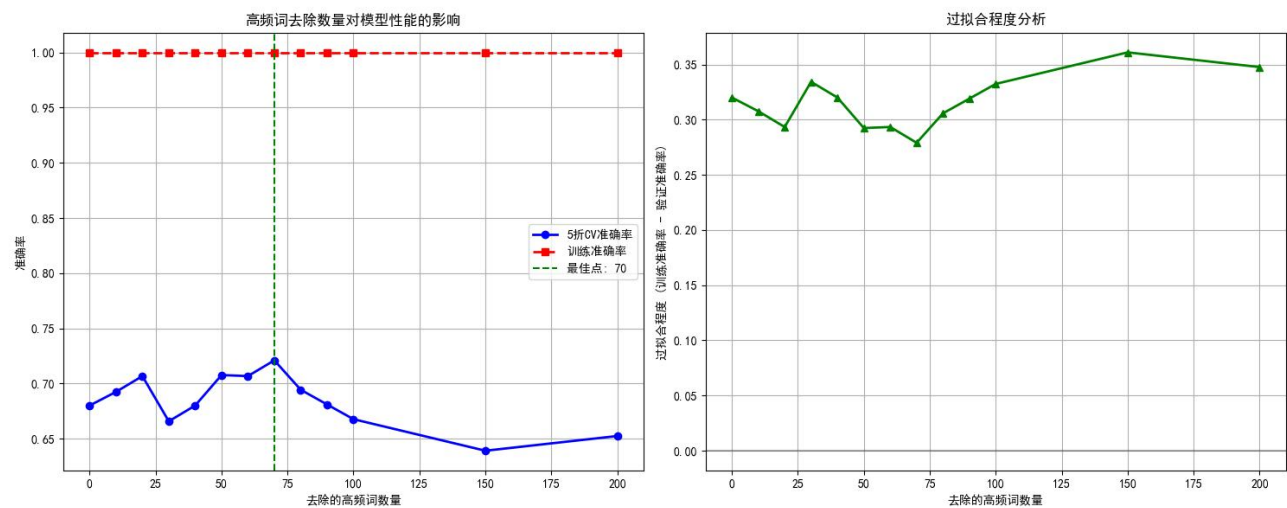
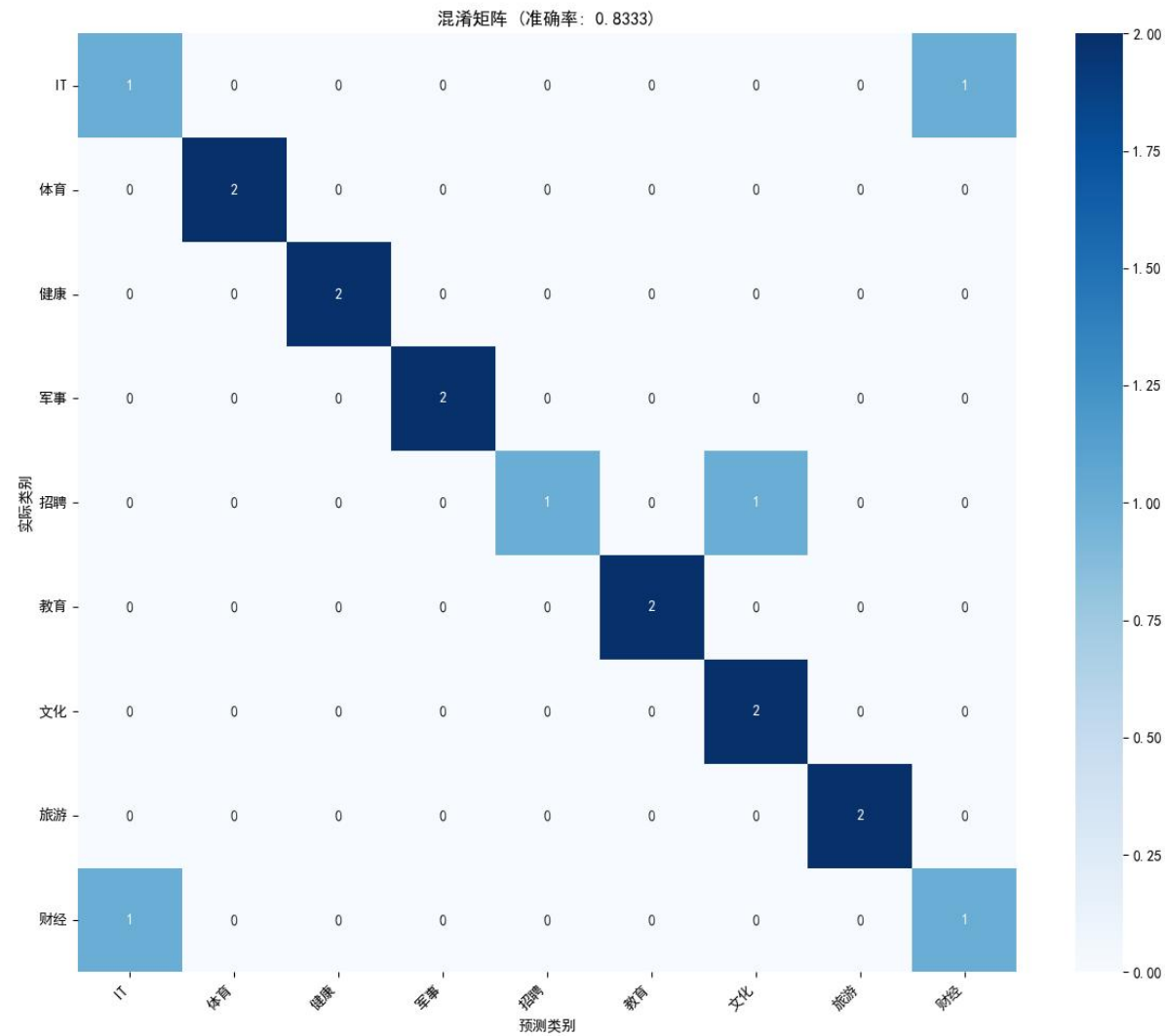
print(f"最终测试集准确率: {test_acc:.4f}")
print(f"结果文件:")
print(f"high_freq_removal_analysis.png (高频词分析)")
print(f"feature_importance.txt (特征重要性)")
print(f"confusion_matrix.png (混淆矩阵)")

if __name__ == "__main__":
    main()

```

2. 运行结果





3.思考题

要确定去除多少高频词是合适的，应该采用基于实验验证的科学方法，而不是随意设定一个固定值。以下是具体的解决方案：

1. 网格搜索 + 交叉验证法：

定义候选的高频词去除数量范围（例如：[0, 10, 20, 50, 100, 150, 200, 300]）

将数据集划分为训练集和验证集

对每个候选数量，使用训练集构建特征矩阵并训练模型

在验证集上评估模型性能（准确率、F1值等指标）

选择使验证集性能最佳的去除数量

2. 交叉验证优化：

使用K折交叉验证（如5折或10折）来减少结果的方差

对每个候选去除数量，计算K次验证的平均性能和标准差

选择平均性能最高且标准差较小的参数

3. 学习曲线分析：

观察随着去除数量增加，训练集和验证集性能的变化趋势

寻找性能饱和点，即继续去除更多高频词不再显著提升性能的点

避免过拟合：当训练集性能远高于验证集时，可能去除的词过多

4. 特征重要性分析：

分析高频词对分类任务的实际贡献

通过信息增益、卡方检验等方法评估每个词的重要性

根据词的重要性和频率综合判断哪些词应该被去除

5. 多指标评估：

不仅看准确率，还要考虑精确率、召回率、F1值等

对于不平衡数据集，特别关注各类别的单独表现

考虑模型的训练时间和内存消耗

6. 最终验证：

在独立的测试集上验证选定参数的最终效果

确保模型的泛化能力

这种方法通过实验数据驱动的方式找到最优参数，既保证了模型性能，又避免了主观臆断，是最科学可靠的解决方案。

4 实验总结与讨论

1. 总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题。

2. 讨论最终得到的结果是否合理、是否达到题目要求；分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法。

通过本次实验，我深入理解了朴素贝叶斯算法在不同场景下的应用特点。朴素贝叶斯非常适合处理高维稀疏数据（如文本向量），在垃圾邮件过滤和新闻分类中表现出色。其假设特征独立的特性虽然在现实中不完全成立，但在文本分类中仍能取得很好的效果，且计算效率高。在侮辱性言论过滤这种特征维度较低、逻辑相对简单的任务中，朴素贝叶斯能够快速学习并达到较高的准确率，适合处理此类“是/否”二分类问题。

在新闻分类任务中，读取SogouC数据集时遇到了中文乱码。通过尝试多种编码格式（如gbk, utf-8, latin1）并捕获异常，最终成功读取了所有文档。中文新闻词汇量巨大，直接向量化会导致维度灾难。通过引入停用词表（stopwords_cn.txt）去除无意义虚词，并结合去除高频词（如“新闻”、“报道”等）的策略，有效降低了特征维度，提升了训练速度。在计算多个概率乘积时，结果可能趋近于0导致下溢。通过将乘法转换为对数加法（np.log），解决了这一数值计算问题。

掌握了完整的文本分类流水线：从数据清洗、中文分词（jieba）、特征提取（词袋模型）到模型训练与评估。理解了特征工程对模型性能的决定性作用，尤其是停用词过滤和高频词去除对提升分类精度的重要性。学会了使用sklearn进行模型评估，通过混淆矩阵和分类报告分析模型的优缺点。

本次实验的最终结果合理且达到了题目要求。基于朴素贝叶斯的原理实现，能够有效区分侮辱性/非侮辱性言论以及垃圾/正常邮件，准确率较高。在经过特征选择（去高频词、去停用词）后，模型在测试集上表现出了较好的泛化能力。混淆矩阵显示，大部分新闻都能被正确归类，说明朴素贝叶斯能够捕捉到不同类别新闻的词汇特征差异。

尽管结果总体良好，但仍存在一定的分类误差，主要原因如下：朴素贝叶斯假设所有词语之间相互独立，忽略了上下文语义（如“不”+“好”=“坏”）。这可能导致部分语义复杂的句子被错误分类。

实验中通过网格搜索确定去除前N个高频词，但有些高频词可能对分类有贡献（如财经新闻中的“股票”）。简单的去除可能导致信息丢失。词袋模型（Bag of Words）丢失了词语的顺序信息，无法处理同义词或一词多义的情况。

解决方法与改进方向：在向量化时使用N-gram（如bigram）模型，捕捉相邻词语之间的关联，缓解特征独立性假设带来的误差。不单纯依赖词频，可以结合卡方检验（Chi-Square）或信息增益（Information Gain）来选择对分类最有贡献的特征，而不是简单地去除高频词。对于中文新闻分类，可以尝试使用基于深度学习的模型（如TextCNN、BERT），这些模型能更好地捕捉语义信息和上下文依赖，进一步提升分类准确率。使用Word2Vec或TF-IDF代替简单的0/1词袋模型，能更好地反映词语的重要程度。